

Failure (respecting a real rate limit) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `retry_backoff_practice.py` (**Failure section**): a real API doesn't just fail randomly — a 429 often comes with a `Retry-After` header telling you exactly how long to wait, and ignoring it makes the rate limit worse for you and everyone sharing your API key. This exercise builds the one function that reads that header correctly, with a safe fallback when it's missing or malformed. **If not:** the Build Task's `http_client.py` would fall back to blind exponential backoff even when the server explicitly says how long to wait — slower to recover, and a worse citizen on a shared API key.

Basic Version

Approach 1 — the direct way

```
# retry_backoff_practice.py – Failure section
class FakeResponse:
    def __init__(self, status_code, headers):
        self.status_code = status_code
        self.headers = headers

def decide_wait_seconds(response, attempt):
    retry_after = response.headers.get("Retry-After")
    if retry_after is not None:
        return int(retry_after)
    return 2 ** attempt

# test: server told us how long to wait
r1 = FakeResponse(429, {"Retry-After": "2"})
print(decide_wait_seconds(r1, attempt=0))    # 2

# test: server didn't say, fall back to backoff
r2 = FakeResponse(429, {})
print(decide_wait_seconds(r2, attempt=3))    # 8
```

Expected output:

```
2  
8
```

This is correct and covers both cases. It's missing type hints and doesn't guard against a malformed `Retry-After` value (like a non-numeric string) — fine for a first pass, worth fixing once you know it's a real risk.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a dataclass fake, and a safe conversion

```
# retry_backoff_practice.py – Failure section
from dataclasses import dataclass, field

@dataclass
class FakeResponse:
    status_code: int
    headers: dict[str, str] = field(default_factory=dict)

def decide_wait_seconds(response: FakeResponse, attempt: int) -> int:
    """Respect Retry-After if the server sent one, else fall back to backoff."""
    retry_after = response.headers.get("Retry-After")
    if retry_after is not None:
        try:
            return int(retry_after)
        except ValueError:
            pass # malformed header – fall through to backoff instead of crashing
    return 2 ** attempt

if __name__ == "__main__":
    told_to_wait = FakeResponse(status_code=429, headers={"Retry-After": "2"})
    print(decide_wait_seconds(told_to_wait, attempt=0)) # 2

    no_header = FakeResponse(status_code=429, headers={})
    print(decide_wait_seconds(no_header, attempt=3)) # 8

    bad_header = {"Retry-After": "soon"}
    malformed_header = FakeResponse(status_code=429, headers=bad_header)
    print(decide_wait_seconds(malformed_header, attempt=2)) # 4, falls back
```

Expected output:

2
8

4

Difference from Basic: using a `@dataclass` for `FakeResponse` gives you a typed, self-documenting test double with almost no code (this is the pattern from Doc01's Build Task Intermediate — reused here, not re-explained). Wrapping the `int(retry_after)` conversion in its own `try/except ValueError` handles a malformed or unexpected header value — a genuinely rate-limited API is exactly the situation where you don't want one more edge case crashing your retry logic instead of just falling back sensibly.

Approach 2 — the HTTP-date form, a hard cap, and the parsing split out so it's independently testable

Two more real gaps: the HTTP spec allows `Retry-After` to be an HTTP-date instead of a plain number, and a server's number should never be trusted blindly — a buggy or malicious server could say "wait 999999999 seconds."

```
# retry_backoff_practice.py – Failure section
from datetime import datetime, timezone
from email.utils import parsedate_to_datetime

MAX_RETRY_AFTER_SECONDS = 60.0

def parse_retry_after(value: str) -> float | None:
    """Return seconds to wait from a Retry-After value, or None if it
    can't be parsed in either the numeric or HTTP-date form."""
    # why: pulled out on its own so the trickiest part (two legal header
    # formats) is testable with plain strings, no fake response needed.
    try:
        # how: the plain-number form, e.g. "2"
        return float(value)
    except ValueError:
        # when: falls through to try the date form instead of crashing here
        pass

    try:
        # how: the HTTP-date form, e.g. "Wed, 21 Oct 2026 07:28:00 GMT"
        target_time = parsedate_to_datetime(value)
        seconds_left = (target_time - datetime.now(timezone.utc)).total_seconds()
        return max(seconds_left, 0.0)
    except (TypeError, ValueError):
        # when: neither form matched – caller falls back to backoff
        return None
```

```
def decide_wait_seconds(headers: dict[str, str], attempt: int) -> float:
    # why: respects a real Retry-After header when the server sends one,
    # caps it so a malicious/buggy server can't stall the client for years.
    retry_after = headers.get("Retry-After")
    wait = parse_retry_after(retry_after) if retry_after is not None else None
    if wait is None:
        # when: no header, or one that couldn't be parsed – fall back to
        # the same exponential backoff used everywhere else in this document.
        wait = float(2 ** attempt)
    # how: never trust a server's number past this cap
    return min(wait, MAX_RETRY_AFTER_SECONDS)

# parse_retry_after can be tested directly, with plain strings, no fake response
# needed
print(parse_retry_after("2"))                                # 2.0
print(parse_retry_after("not a number or a date"))           # None
print(decide_wait_seconds({"Retry-After": "999999999"}, 0))  # 60.0
```

Expected output:

```
2.0
None
60.0
```

The last line is the important one: a server claiming you should wait for over 31 years gets capped at 60 seconds instead of being honored literally.

Difference from Approach 1: Approach 1 trusts `Retry-After` to always be a plain integer string from a well-behaved server — a date-formatted header or a deliberately huge number either crashes it or makes it wait an absurd amount of time. Approach 2 splits the parsing (`parse_retry_after`) from the capping-and-fallback decision (`decide_wait_seconds`), so the trickiest part — turning a raw header string into a number of seconds, in either of its two legal forms — can be tested directly with plain string inputs, the same "pull the risky logic into its own pure function" idea from the timeout/retry exercise's `compute_backoff_delay` .

Which one should you actually write? For `http_client.py` in this document's Build Task, Approach 2's split is worth it — `parse_retry_after()` is exactly the kind of function you want a handful of quick unit tests for (a plain number, an HTTP-date, garbage, `None`), and testing it doesn't require constructing a fake response object at all. The hard cap (`MAX_RETRY_AFTER_SECONDS`) is worth keeping in any version of this code that talks to a server you don't fully control — which, for an external API, is always. It's also exactly the kind of number that belongs in `config.py` (Doc01)

rather than hardcoded here — a cap you might reasonably want to raise or lower per environment without editing this function.